

TP1 - Powershell

Trouver la commande powershell pour :

→ Afficher les dossiers et fichiers d'un répertoire.

Get-ChildItem -Path "C:"

```
PS C:\> Get-ChildItem -Path "C:"
Directory: C:\
Name            Mode                LastWriteTime         Length Name
-----
[...]
```

→Ajouter un nouvel utilisateur au système

New-LocalUser -Name "lisa" -Password (ConvertTo-SecureString "MotDePasseSecurisé" -AsPlainText -Force) -FullName "Lisa" -Description "Utilisateur local"

```
PS C:\Windows\system32> New-LocalUser -Name "lisa" -Password (ConvertTo-SecureString "MotDePasseSecurisé" -AsPlainText -Force) -FullName "Lisa" -Description "Utilisateur local"
Name Description
----
lisa lisa      Utilisateur local

PS C:\Windows\system32>

Selecter une des commandes:
Microsoft Windows [version 10.0.19042.1103]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\admin> net user

Comptes d'utilisateurs de l'NT-LV-Wild

Nom          Administrateur      De-faut Account
-----
Invité       lisa               lisa
La commande a été terminée correctement.
```

→Lister les processus en cours d'exécution sur le système.

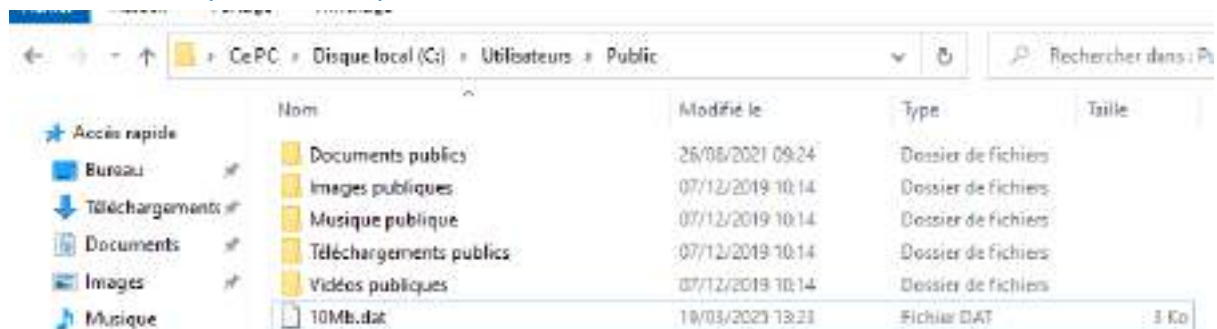
Get-Process

```
PS C:\Windows\system32> Get-Process
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
424	24	15184	32748	0,20	7404	1	ApplicationFrameHost
73	5	2328	4360	0,02	4928	1	cmd
117	6	1060	3400	0,02	2340	0	CompatTelRunner
312	15	3420	9900	0,05	2636	0	CompatTelRunner
270	13	7460	21212	0,20	4984	1	conhost
161	10	6688	7284	0,02	7016	0	conhost
264	14	5632	27192	6,34	7428	1	conhost
573	21	1760	5744	0,55	488	0	csrss
417	16	1840	5228	2,70	580	1	csrss
430	16	4016	20196	0,73	4268	1	ctfmon
211	16	3272	12048	0,13	1276	0	dllhost
252	23	5396	13488	0,16	6060	1	dllhost
997	44	52432	84960	4,50	584	1	dwm
2186	80	53660	117148	4,98	5380	1	explorer
32	5	1576	4192	0,06	872	1	fontdrvhost
32	5	1280	3292	0,00	880	0	fontdrvhost
0	0	60	8		0	0	Idle

→Télécharger un fichier à partir d'Internet et le stocker sur le disque local.

`Invoke-WebRequest -Uri "http://ovh.net/files/10Mb.dat" -OutFile "C:\Users\Public\10Mb.dat"`



The screenshot shows a Windows File Explorer window with the address bar set to 'Ce PC > Disque local (C:) > Utilisateurs > Public'. The left sidebar shows 'Accès rapide' with links to Bureau, Téléchargements, Documents, Images, and Musique. The main area displays a list of folders and files:

Nom	Modifié le	Type	Taille
Documents publics	26/06/2021 09:24	Dossier de fichiers	
Images publiques	07/12/2019 10:14	Dossier de fichiers	
Musique publique	07/12/2019 10:14	Dossier de fichiers	
Téléchargements publics	07/12/2019 10:14	Dossier de fichiers	
Vidéos publiques	07/12/2019 10:14	Dossier de fichiers	
10Mb.dat	10/03/2023 18:23	Fichier DAT	1 Ko

→Afficher les informations système telles que la mémoire disponible, le nombre de processeurs, etc.

Get-ComputerInfo

```
PS C:\Windows\system32> Get-ComputerInfo
```

WindowsBuildLabEx	: 19H041.1.amd64fre.vb_release.191206-1400
WindowsCurrentVersion	: 6.3
WindowsEditionId	: Professional
WindowsInstallationType	: Client
WindowsInstallDateFromRegistry	: 26/06/2021 07:24:09
WindowsProductId	: 00330-80000-00000-A0923
WindowsProductName	: Windows 10 Pro
WindowsRegisteredOrganization	:
WindowsRegisteredOwner	: admin
WindowsSystemRoot	: C:\Windows
WindowsVersion	: 20H2
BiosCharacteristics	: {4, 7, 15, 16, ...}
BiosBiosVersion	: {VBOX - 1, VirtualBox, EDK II - 10000}
BiosFirmwareManufacturer	:

Get-ComputerInfo -Property "*version"

```
PS C:\Windows\system32> Get-ComputerInfo -Property "*version"

WindowsCurrentVersion      : 6.3
WindowsVersion              : 2009
BiosBIOSVersion             : {VBOX - 1, VirtualBox, EDK II - 10000}
BiosEmbeddedControllerMajorVersion : 110
BiosEmbeddedControllerMinorVersion : 111
BiosSMBIOSBIOSVersion       : VirtualBox
BiosSMBIOSMajorVersion      : 2
BiosSMBIOSMinorVersion      : 5
BiosSystemBiosMajorVersion  : 105
BiosSystemBiosMinorVersion  : 110
BiosVersion                 : VBOX - 1
OsVersion                   : 10.0.19042
OsCSDVersion                :
OsServicePackMajorVersion   : 0
OsServicePackMinorVersion   : 0
```

Get-CimInstance Win32_OperatingSystem | Select-Object FreePhysicalMemory

```
PS C:\Windows\system32> Get-CimInstance Win32_OperatingSystem | Select-Object FreePhysicalMemory

FreePhysicalMemory
-----
4698356
```

→ Exécuter des commandes réseau telles que le ping ou nslookup.

Test-Connection 172.31.25.9 -Count 4

```
PS C:\Windows\system32> Test-Connection 172.31.25.9 -Count 4
>>

Source      Destination    IPv4Address    IPv6Address    Bytes    Time(ms)
-----
RT-MW-U10   172.31.25.9    172.31.25.9    -              32       0
RT-MW-U10   172.31.25.9    172.31.25.9    -              32       1
RT-MW-U10   172.31.25.9    172.31.25.9    -              32       0
RT-MW-U10   172.31.25.9    172.31.25.9    -              32       0

PS C:\Windows\system32>
```

→ Modifier les paramètres du système tels que le planificateur de tâches et les services.

afficher les tâches : `Get-ScheduledTask`

```
\Microsoft\Windows\Windows Defender\ Windows Defender Cleanup Ready
\Microsoft\Windows\Windows Defender\ Windows Defender Scheduled Scan Ready
\Microsoft\Windows\Windows Defender\ Windows Defender Verification Ready
\Microsoft\Windows\Windows Error Reporting\ QueueReporting Ready
\Microsoft\Windows\Windows Filtering Platform\ BfeOnServiceStartTypeChange Ready
\Microsoft\Windows\Windows Media Sharing\ UpdateLibrary Ready
\Microsoft\Windows\WindowsColorSystem\ Calibration Loader Ready
\Microsoft\Windows\WindowsUpdate\ Refresh Group Policy Cache Ready
\Microsoft\Windows\WindowsUpdate\ Scheduled Start Ready
\Microsoft\Windows\Wininet\ CacheTask Running
\Microsoft\Windows\WlanSvc\ CDSSync Ready
\Microsoft\Windows\WOF\ WIM-Hash-Management Ready
\Microsoft\Windows\WOF\ WIM-Hash-Validation Ready
\Microsoft\Windows\Work Folders\ Work Folders Logon Synchroniza... Ready
\Microsoft\Windows\Work Folders\ Work Folders Maintenance Work Ready
\Microsoft\Windows\Workplace Join\ Automatic-Device-Join Disabled
\Microsoft\Windows\Workplace Join\ Device-Sync Disabled
\Microsoft\Windows\Workplace Join\ Recovery-Check Disabled
\Microsoft\Windows\WwanSvc\ NotificationTask Ready
\Microsoft\Windows\WwanSvc\ OobeDiscovery Ready
\Microsoft\XblGameSave\ XblGameSaveTask Ready

PS C:\Windows\system32> Get-ScheduledTask_
```

et crée une nouvelle tâche auto:

```
$action = New-ScheduledTaskAction -Execute "powershell.exe" -Argument "-File
C:\Scripts\mon_script.ps1"
```

```
$trigger = New-ScheduledTaskTrigger -Daily -At "12:00PM"
```

```
Register-ScheduledTask -TaskName "MaNouvelleTache" -Action $action -Trigger $trigger
-Description "Exécute un script tous les jours à 12h" -User "SYSTEM" -RunLevel Highest
```

```
PS C:\Windows\system32> Get-ScheduledTask -TaskName "MaNouvelleTache"
PS C:\Windows\system32> Get-ScheduledTask -TaskName "MaNouvelleTache"
PS C:\Windows\system32> Register-ScheduledTask -TaskName "MaNouvelleTache" -Action $action -Trigger $trigger -Description "Exécute un script tous les jours à 12h" -User "SYSTEM" -RunLevel Highest
```

TaskName	TaskName	Status
MaNouvelleTache	MaNouvelleTache	Ready

→ Gérer les utilisateurs et les groupes avec PowerShell.

`Get-LocalUser`

`Add-LocalGroupMember -Group "Administrateurs" -Member "lisa"`

```
PS C:\Windows\system32> Add-LocalGroupMember -Group "Administrateurs" -Member "lisa"
PS C:\Windows\system32> Get-LocalGroupMember -Group "Administrateurs"
>>
```

ObjectClass	Name	PrincipalSource
Utilisateur	RT-MV-W10\admin	Local
Utilisateur	RT-MV-W10\Administrateur	Local
Utilisateur	RT-MV-W10\lisa	Local

→ Afficher des données de la base de registre, telles que les clés de registre et les valeurs.

`Get-ItemProperty -Path "HKLM:\Software\Microsoft\Windows\CurrentVersion"`

```
PS C:\Windows\system32> Get-ItemProperty -Path "HKCU\Software\Microsoft\Windows\CurrentVersion\Program Files"

ProgramFilesDir       : C:\Program Files
CommonFilesDir        : C:\Program Files\Common Files
ProgramFilesDir (x86) : C:\Program Files (x86)
CommonFilesDir (x86)  : C:\Program Files (x86)\Common Files
CommonW6432Dir       : C:\Program Files\Common Files
DevicePath            : C:\Windows\inf
MediaPathUnexpanded   : C:\Windows\Media
ProgramFilesPath       : C:\Program Files
ProgramW6432Dir       : C:\Program Files
SP_ConfigureProgramName : Set Program Access and Defaults
SP_GoiesName           : Goies
PSPath                : Microsoft.PowerShell.Core\Registry::HKCU\Software\Microsoft\Windows\CurrentVersion
PSParentPath           : Microsoft.PowerShell.Core\Registry::HKCU\Software\Microsoft\Windows
PSChildName            : CurrentVersion
PSDrive                : HKCU
PSProvider              : Microsoft.PowerShell.Core\Registry
```

Registre

Ecrire un script qui effectuera les actions ci-dessous, dans la ruche
HKEY_CURRENT_USER de la bdr :

- Créer une clef nommée Powershell
- Dans la clef Powershell, créer une valeur de type DWORD nommée Value1 ayant pour valeur 1
- Dans la clef Powershell, créer une valeur de type texte nommée Value2 ayant pour valeur la chaîne « Powershell, c'est facile ».
- Trouvez un moyen d'ignorer les erreurs d'exécution vous permettant d'exécuter votre script plusieurs fois sans erreurs.

```
$path = "HKCU:\Software\Powershell"
```

```
# Vérifier si la clé existe déjà, sinon la créer
```

```
if (-not (Test-Path $path)) {
    New-Item -Path $path -Force | Out-Null
    Write-Output "Clé 'Powershell' créée avec succès."
}
```

```
# Ajouter la valeur DWORD "Value1" uniquement si elle n'existe pas
```

```
if (-not (Get-ItemProperty -Path $path -Name "Value1" -ErrorAction SilentlyContinue)) {
    New-ItemProperty -Path $path -Name "Value1" -Value 1 -PropertyType DWORD
    -Force | Out-Null
    Write-Output "Valeur DWORD 'Value1' ajoutée avec succès."
} else {
    Write-Output "La valeur 'Value1' existe déjà."
}
```

```
# Ajouter la valeur texte "Value2" uniquement si elle n'existe pas
```

```

if (-not (Get-ItemProperty -Path $path -Name "Value2" -ErrorAction SilentlyContinue)) {
    New-ItemProperty -Path $path -Name "Value2" -Value "Powershell, c'est facile"
    -PropertyType String -Force | Out-Null
    Write-Output "Valeur texte 'Value2' ajoutée avec succès."
} else {
    Write-Output "La valeur 'Value2' existe déjà."
}

```

```

script.ps1 X
1 $path = "HKCU\Software\Powershell"
2
3 # Vérifier si la clé existe déjà, sinon la créer
4 if (-not (Test-Path $path)) {
5     New-Item -Path $path -Force | Out-Null
6     Write-Output "Clé 'Powershell' créée avec succès."
7 }
8
9 # Ajouter la valeur DWORD 'Value1' uniquement si elle n'existe pas
10 if (-not (Get-ItemProperty -Path $path -Name 'Value1' -ErrorAction SilentlyContinue)) {
11     New-ItemProperty -Path $path -Name 'Value1' -Value 1 -PropertyType DWORD -Force | Out-Null
12     Write-Output "Valeur DWORD 'Value1' ajoutée avec succès."
13 } else {
14     Write-Output "La valeur 'Value1' existe déjà."
15 }
16
17 # Ajouter la valeur texte 'Value2' uniquement si elle n'existe pas
18 if (-not (Get-ItemProperty -Path $path -Name 'Value2' -ErrorAction SilentlyContinue)) {
19     New-ItemProperty -Path $path -Name 'Value2' -Value "Powershell, c'est facile" -PropertyType String -Force | Out-Null
20     Write-Output "Valeur texte 'Value2' ajoutée avec succès."
21 } else {
22     Write-Output "La valeur 'Value2' existe déjà."
23 }
24
25

```

```

PS C:\Users\admin\Desktop> C:\Users\admin\Desktop\script.ps1
Impossible de charger le fichier C:\Users\admin\Desktop\script.ps1, car l'exécution de scripts est désactivée sur ce
système. Pour plus d'informations, consultez about_Execution_Policies à l'adresse
https://go.microsoft.com/fwlink/?LinkID=135170.
+ CategoryInfo          : Erreur de sécurité ( ( ) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess

PS C:\Users\admin\Desktop> cd C:\Users\admin\Desktop

PS C:\Users\admin\Desktop> .\script.ps1
.\script.ps1 : Impossible de charger le fichier C:\Users\admin\Desktop\script.ps1, car l'exécution de scripts est
désactivée sur ce système. Pour plus d'informations, consultez about_Execution_Policies à l'adresse
https://go.microsoft.com/fwlink/?LinkID=135170.
Au caractère Ligne:1 : 1
+ .\script.ps1
+ ~~~~~
+ CategoryInfo          : Erreur de sécurité ( ( ) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess

PS C:\Users\admin\Desktop> C:\Users\admin\Desktop\script.ps1
Clé 'Powershell' créée avec succès.
Valeur DWORD 'Value1' ajoutée avec succès.
Valeur texte 'Value2' ajoutée avec succès.

PS C:\Users\admin\Desktop> C:\Users\admin\Desktop\script.ps1
La valeur 'Value1' existe déjà.
La valeur 'Value2' existe déjà.

PS C:\Users\admin\Desktop> C:\Users\admin\Desktop\script.ps1
La valeur 'Value1' existe déjà.
La valeur 'Value2' existe déjà.

PS C:\Users\admin\Desktop> C:\Users\admin\Desktop\script.ps1
La valeur 'Value1' existe déjà.
La valeur 'Value2' existe déjà.

PS C:\Users\admin\Desktop> C:\Users\admin\Desktop\script.ps1
La valeur 'Value1' existe déjà.
Valeur texte 'Value2' ajoutée avec succès.

```

Modules et KB

Dans l'interpréteur de commandes Powershell :

→ Trouver une commande permettant d'avoir la liste des mises à jours systèmes installées sur la machine courante.

[Get-HotFix](#)

```
PS C:\Windows\system32> Get-HotFix
```

Source	Description	HotFixID	InstalledBy	InstalledOn
RT-MV-W10	Update	KB5004331	AUTORITE NT\Système	26/08/2021 00:00:00
RT-MV-W10	Update	KB4562830	AUTORITE NT\Système	26/08/2021 00:00:00
RT-MV-W10	Security Update	KB4570334		27/09/2020 00:00:00
RT-MV-W10	Security Update	KB4577266		09/10/2020 00:00:00
RT-MV-W10	Update	KB4577586	AUTORITE NT\Système	26/08/2021 00:00:00
RT-MV-W10	Security Update	KB4580325		09/10/2020 00:00:00
RT-MV-W10	Update	KB5033052	AUTORITE NT\Système	19/03/2025 00:00:00
RT-MV-W10	Security Update	KB5005033	AUTORITE NT\Système	26/08/2021 00:00:00
RT-MV-W10	Security Update	KB5005260	AUTORITE NT\Système	26/08/2021 00:00:00

Dans Visual Studio Code :

→ Ecrire une fonction de module Test-KBInstalled qui prends en Paramètre le nom du KB et renvoi un boolean

(hormis le passage de paramètre, la fonction peut être écrite en une ligne)

Fonctionnement recherché (dans une console) :

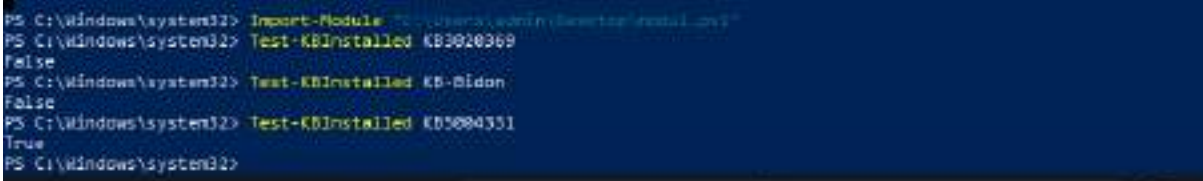
```
PS C:\IUT> Import-Module mon-module.psm1
PS C:\IUT> Test-KBInstalled KB-Bidon
False
PS C:\IUT> Test-KBInstalled KB3020369
True
```

ASTUCE DEBUG : Utilisez Get-Module et Remove-Module

```
function Test-KBInstalled {
    param (
        [string]$KBID
    )
    return (Get-HotFix -Id $KBID -ErrorAction SilentlyContinue) -ne $null
}

# Vérifie si le KB est installé en utilisant Get-HotFix
# -ErrorAction SilentlyContinue permet d'éviter l'affichage d'une erreur si le KB n'existe pas
```

```
PS C:\Windows\system32> Import-Module "C:\Users\admin\Desktop\test1.ps1"
PS C:\Windows\system32> Test-KBInstalled KB3020369
False
PS C:\Windows\system32> Test-KBInstalled KB-Bidon
False
PS C:\Windows\system32> Test-KBInstalled KB5004351
True
PS C:\Windows\system32>
```



```
modul.ps1
> Users > admin > Desktop > modul.ps1
1 function Test-KBInstalled {
2     param (
3         [string]$KBID
4     )
5     return (Get-HotFix -Id $KBID -ErrorAction SilentlyContinue) -ne $null
6 }
7
8 # Vérifie si le KB est installé en utilisant Get-HotFix
9 # -ErrorAction SilentlyContinue permet d'éviter l'affichage d'une erreur si le KB n'existe pas
10
11
12
```

IO

Soit deux fichiers textes comme suit contenant des listes d'ordinateurs (dont PC1 en commun). Pour ceux qui sont à l'aise, créer ces fichiers texte via le shell.

List1.txt

PC1

PC2

PC5

List2.txt

PC1

PC3

PC4

Ecrire un script PowerShell qui donne le rapport suivant :

Rapport de comparaison le 17/02/2022

Ces ordinateurs ne sont pas dans .\List1.txt

PC3

PC4

Ces ordinateurs ne sont pas dans .\List2.txt

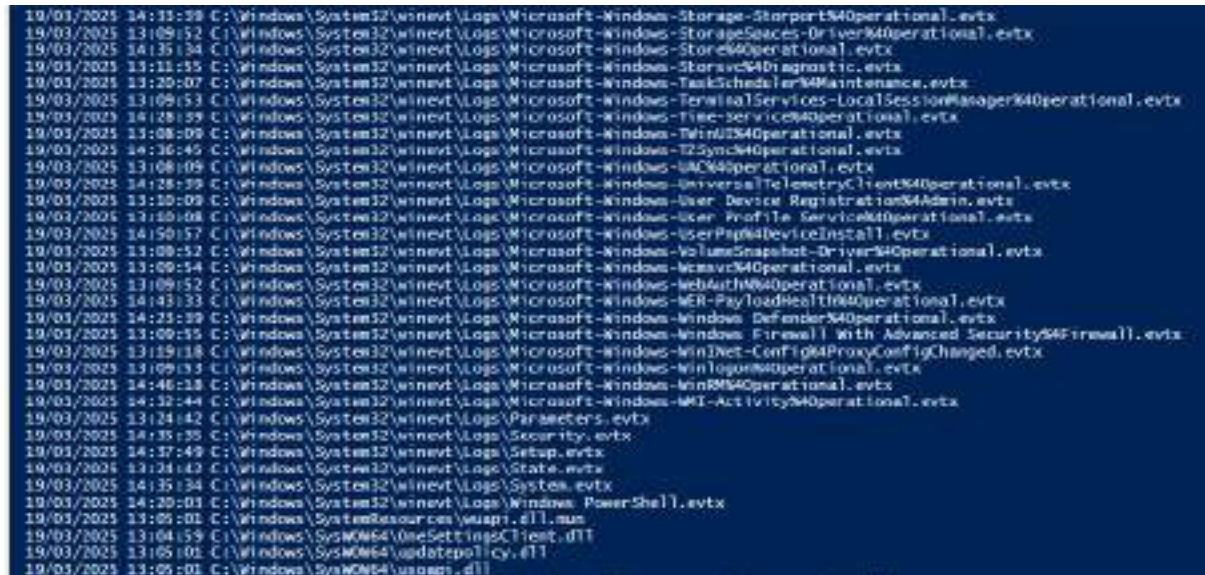
PC2

PC5

PS C:\ADN\AIS02\TP2\


```
Get-ChildItem -Path C:\ -Recurse -File | Where-Object {($_.LastWriteTime -ge (Get-Date).AddDays(-7)) -and ($_.Extension -ne ".tmp")} | Select-Object LastWriteTime, FullName
```

Exécute le script dans powershell
.*recherche_fichiers.ps1*



```
19/03/2025 14:11:38 C:\Windows\System32\winevt\Logs\Microsoft-Windows-Storage-StorageSpaces-Driver\40operational.evtx
19/03/2025 13:09:52 C:\Windows\System32\winevt\Logs\Microsoft-Windows-StorageSpaces-Driver\40operational.evtx
19/03/2025 14:13:34 C:\Windows\System32\winevt\Logs\Microsoft-Windows-Storage\40operational.evtx
19/03/2025 13:11:55 C:\Windows\System32\winevt\Logs\Microsoft-Windows-Storage\40diagnostic.evtx
19/03/2025 13:29:07 C:\Windows\System32\winevt\Logs\Microsoft-Windows-TaskScheduler\40Maintenance.evtx
19/03/2025 13:09:53 C:\Windows\System32\winevt\Logs\Microsoft-Windows-TerminalServices-LocalSessionManager\40operational.evtx
19/03/2025 14:12:59 C:\Windows\System32\winevt\Logs\Microsoft-Windows-Time-Service\40operational.evtx
19/03/2025 13:08:09 C:\Windows\System32\winevt\Logs\Microsoft-Windows-Timedate\40operational.evtx
19/03/2025 14:16:45 C:\Windows\System32\winevt\Logs\Microsoft-Windows-T2Sync\40operational.evtx
19/03/2025 13:08:09 C:\Windows\System32\winevt\Logs\Microsoft-Windows-UAC\40operational.evtx
19/03/2025 14:18:39 C:\Windows\System32\winevt\Logs\Microsoft-Windows-UniversalTelemetryClient\40operational.evtx
19/03/2025 13:10:09 C:\Windows\System32\winevt\Logs\Microsoft-Windows-User-Device-Registration\40Admin.evtx
19/03/2025 13:10:09 C:\Windows\System32\winevt\Logs\Microsoft-Windows-User-Profile-Service\40operational.evtx
19/03/2025 14:10:08 C:\Windows\System32\winevt\Logs\Microsoft-Windows-User-Profile-Service\40install.evtx
19/03/2025 14:10:57 C:\Windows\System32\winevt\Logs\Microsoft-Windows-VolumeSnapshot-Driver\40operational.evtx
19/03/2025 13:09:54 C:\Windows\System32\winevt\Logs\Microsoft-Windows-WcmSvc\40operational.evtx
19/03/2025 13:09:52 C:\Windows\System32\winevt\Logs\Microsoft-Windows-WebAuthN\40operational.evtx
19/03/2025 14:13:33 C:\Windows\System32\winevt\Logs\Microsoft-Windows-WER-PayloadHealth\40operational.evtx
19/03/2025 14:23:39 C:\Windows\System32\winevt\Logs\Microsoft-Windows-Windows-Defender\40operational.evtx
19/03/2025 13:09:35 C:\Windows\System32\winevt\Logs\Microsoft-Windows-Windows-Firewall-With-Advanced-Security\40Firewall.evtx
19/03/2025 13:19:18 C:\Windows\System32\winevt\Logs\Microsoft-Windows-Wininet-Config\40ProxyConfigChanged.evtx
19/03/2025 13:09:33 C:\Windows\System32\winevt\Logs\Microsoft-Windows-Winlogon\40operational.evtx
19/03/2025 14:46:18 C:\Windows\System32\winevt\Logs\Microsoft-Windows-WinRM\40operational.evtx
19/03/2025 14:12:44 C:\Windows\System32\winevt\Logs\Microsoft-Windows-WMI-Activity\40operational.evtx
19/03/2025 13:24:42 C:\Windows\System32\winevt\Logs\Parameters.evtx
19/03/2025 14:15:36 C:\Windows\System32\winevt\Logs\Security.evtx
19/03/2025 14:17:49 C:\Windows\System32\winevt\Logs\Setup.evtx
19/03/2025 13:24:42 C:\Windows\System32\winevt\Logs\State.evtx
19/03/2025 14:15:34 C:\Windows\System32\winevt\Logs\System.evtx
19/03/2025 14:20:01 C:\Windows\System32\winevt\Logs\Windows-PowerShell.evtx
19/03/2025 13:05:01 C:\Windows\System32\winevt\Logs\Windows-SystemResources\40api.dll.mus
19/03/2025 13:04:59 C:\Windows\System32\winevt\Logs\Windows-SystemResources\40api.dll.mus
19/03/2025 13:05:01 C:\Windows\System32\winevt\Logs\Windows-SystemResources\40api.dll.mus
19/03/2025 13:05:01 C:\Windows\System32\winevt\Logs\Windows-SystemResources\40api.dll.mus
```

Script

Écrire un script qui effectue une opération de synchronisation de fichiers entre deux répertoires.

Contenu du fichier *sync_files.ps1* :

```
param (
    [string]$Source = "C:\Source",
    [string]$Destination = "C:\Destination"
)

# Vérifier si les dossiers existent
if (!(Test-Path $Source)) {
    Write-Host "Le dossier source n'existe pas : $Source" -ForegroundColor Red
    exit
}
if (!(Test-Path $Destination)) {
    Write-Host "Le dossier destination n'existe pas, création en cours..."
    -ForegroundColor Yellow
    New-Item -ItemType Directory -Path $Destination | Out-Null
}

# Synchronisation des fichiers
Write-Host "Synchronisation en cours..." -ForegroundColor Cyan
Robocopy $Source $Destination /E /PURGE /XO
```

```
Write-Host "Synchronisation terminée !" -ForegroundColor Green
```

```
.\sync_files.ps1
```